

AI FRAUD & RISK DETECTION PLATFORM

QA AUDIT REPORT

Functional • Network Graph • Integration • Logic • Security • Performance

Prepared for internship project review

Executive Summary

This report consolidates the QA findings from the supplied OpenCode audit. The original findings, test outcomes, bug IDs, severities, and technical evidence have been preserved; the document has been reformatted for readability and submission.

Metric	Count
A. Total Test Cases	50
B. Passed	18
C. Failed	25
D. Partially Passed	7
E. Not Implemented	5 (covered within failed/partial)

Severity	Count	Bug IDs
F. CRITICAL	2	BUG-001, BUG-002
G. HIGH	8	BUG-003, BUG-004, BUG-005, BUG-006, BUG-007, BUG-008, BUG-009, BUG-010
H. MEDIUM	9	BUG-011, BUG-012, BUG-013, BUG-014, BUG-015, BUG-016, BUG-017, BUG-018, BUG-022
LOW	4	BUG-019, BUG-020, BUG-021, BUG-023

Overall Project Status

NOT READY

The audit identifies four release blockers: dashboard API integration, network graph risk-data integration, non-functional navigation, and missing frontend loading/error/empty states.

A. Functional Testing

#	Feature	Status	Notes
1.1	Create transaction	PARTIALLY WORKING	Backend endpoint exists and works, but frontend FraudDashboard.jsx uses hardcoded mock data — never calls the API
1.2	Transaction history	PARTIALLY WORKING	Backend has pagination/filtering; frontend shows a static table with fake data
1.3	Batch transaction creation	PASS	Backend /batch endpoint functional, silently skips duplicates
1.4	CSV import	PARTIALLY WORKING	Backend exists but has critical bugs (see BUG-003, BUG-004)
1.5	CSV template download	FAIL	Endpoint is POST when it should be GET — semantically wrong for a download

#	Feature	Status	Notes
2.1	Customer risk profile	PARTIALLY WORKING	CustomerRiskProfile model exists; investigation service reads it; no dedicated customer-facing UI
2.2	Suspicious customers list	PARTIALLY WORKING	Backend /dashboard/suspicious-customers exists; frontend shows hardcoded mock data
2.3	Customer search	NOT IMPLEMENTED	No customer search endpoint or UI

#	Feature	Status	Notes
3.1	ML anomaly scoring	PARTIALLY WORKING	Isolation Forest integration exists, falls back to rule-only on missing model
3.2	Rule engine scoring	PARTIALLY WORKING	Rules engine exists but RuleOperator enum comparison against raw strings may silently fail
3.3	Risk score 0–100 range	PARTIALLY WORKING	ML engine normalizes to 0–100; but csv_import.py truncates final_risk_score via int() cast
3.4	Low/Medium/High classification	PARTIALLY WORKING	Thresholds defined (0–30, 31–70, 71–100) but duplicated as magic numbers in 4+ files
3.5	Risk level display (frontend)	FAIL	Frontend is all mock data; network graph has critical data mismatch (see BUG-001)

#	Feature	Status	Notes
4.1	Fraud detection (ML)	PARTIALLY WORKING	Backend scoring works when ML model is loaded; no frontend integration
4.2	Fraud detection (rules)	PARTIALLY WORKING	Business rules exist but duplicated between ml_integration.py and risk_engine.py with different rule sets
4.3	Alert generation on high risk	PASS	Alert created when risk_level == "HIGH" in transaction creation
4.4	Fraud flag display	NOT IMPLEMENTED	No UI shows fraud detection results from the API
#	Feature	Status	Notes
5.1	Alert list with pagination	PASS	Backend endpoint functional
5.2	Alert filtering	PASS	Status, severity, date range, assigned_to filters work
5.3	Alert detail view	PASS	Backend returns full alert data
5.4	Alert status update	PASS	PATCH endpoint updates status, sets resolved_at
5.5	Alert assignment	PASS	POST endpoint assigns alert to user
5.6	Alert investigation notes	PASS	Notes appended with timestamp
5.7	Frontend alert display	FAIL	FraudDashboard.jsx shows hardcoded mock alerts; alertService.js exists but is never called by the UI
#	Feature	Status	Notes
6.1	Investigation detail endpoint	PARTIALLY WORKING	Backend service exists with full investigation logic
6.2	Related transactions	PARTIALLY WORKING	Backend returns related transactions; has func.array_agg() PostgreSQL-specific dependency
6.3	Device/IP/Location info	PARTIALLY WORKING	Backend aggregates from transactions; frontend has no investigation UI
6.4	Risk factors display	PARTIALLY WORKING	Risk flags stored in JSON; no investigation detail UI exists
6.5	AI explanation in investigation	PARTIALLY WORKING	AI integration exists but no fallback for generate_investigation_summary errors
#	Feature	Status	Notes
7.1	AI explanation generation	PARTIALLY WORKING	Gemini integration exists with fallback; no timeout/retry logic
7.2	Investigation summary	PARTIALLY WORKING	Endpoint exists; no error fallback (unlike generate_explanation)
7.3	Q&A functionality	PARTIALLY WORKING	answer_investigation_question exists; no error fallback
7.4	Frontend AI assistant UI	NOT IMPLEMENTED	No AI assistant interface exists in the frontend
#	Feature	Status	Notes
8.1	KPI cards	FAIL	All KPI values are hardcoded in FraudDashboard.jsx — no API calls
8.2	Transaction trends chart	FAIL	Uses hardcoded trendsData array, not from API
8.3	Risk distribution pie chart	FAIL	Uses hardcoded riskData array
8.4	Suspicious customers table	FAIL	Uses hardcoded suspiciousCustomers array
8.5	Alerts panel	FAIL	Uses hardcoded alerts array
8.6	Backend dashboard stats	PASS	All 6 dashboard service methods exist and are wired to endpoints

B. Network Graph QA

#	Test	Status	Evidence
B.1	Backend NetworkService builds graph	PASS	network.py:33-237 — queries customers, transactions, devices, IPs, locations

#	Test	Status	Evidence
B.2	Backend returns NetworkNode/NetworkEdge	PASS	Schema validates response structure
B.3	Frontend fetches graph data	PASS	useNetwork.js calls networkService.getGraph()
B.4	Risk data correctly reflected	CRITICAL FAIL	Backend sends risk_level as string in metadata; frontend reads metadata.risk_score which never exists — risk is always undefined
#	Test	Status	Evidence
B.5	Customer nodes load	PASS	Backend creates nodes with type customer
B.6	Device nodes load	PASS	Backend creates nodes with type device
B.7	IP nodes load	PASS	Backend creates nodes with type ip_address
B.8	Transaction nodes load	PASS	Backend creates nodes with type transaction
B.9	Location nodes load	PASS	Backend creates nodes with type location
B.10	Customer → Device relationship	PASS	Edge type uses_device created
B.11	Customer → IP relationship	PASS	Edge type uses_ip created
B.12	Customer → Transaction relationship	PASS	Edge type made_transaction created
B.13	Transaction → Location relationship	PASS	Edge type from_location created
#	Test	Status	Evidence
B.14	Risk scores displayed on nodes	CRITICAL FAIL	FraudNode.jsx:22-28 reads data.risk which is undefined for API data
B.15	Risk color coding (Low/Med/High)	CRITICAL FAIL	theme.js:27-35 riskTone() receives undefined — no color applied
B.16	Risk filter works	CRITICAL FAIL	FraudNetwork.jsx:99-103 filters by n.risk which is undefined — all nodes pass filter
B.17	Low/Medium/High risk indication correct	CRITICAL FAIL	Cannot be verified because risk data is broken end-to-end
#	Test	Status	Evidence
B.18	Search works	PARTIALLY WORKING	Search filters by label text; works on mock data
B.19	Node selection works	PASS	onNodeClick sets selectedEntity
B.20	Connected-node highlighting	PARTIALLY WORKING	highlightedNodes/highlightedEdges computed; but depends on correct edges
B.21	Details panel shows info	PARTIALLY WORKING	Panel renders; shows entity type/value but risk info is missing
B.22	Zoom works	PASS	ReactFlow default controls
B.23	Pan works	PASS	ReactFlow default controls
B.24	Minimap works	PASS	<MiniMap /> rendered
B.25	Graph doesn't break with missing data	PASS	Fallback { nodes: [], edges: [] } in useNetwork.js:5-6
B.26	Graph doesn't break with many nodes	PASS	limit parameter caps at 2000; ReactFlow handles large datasets
B.27	useMemo missing nodes dependency	BUG	FraudNetwork.jsx:99 — filteredNodes doesn't list nodes in deps — stale closure risk

C. API / Data Integration Testing

#	Test	Status	Evidence
C.1	API client sends requests	PARTIALLY WORKING	apiClient.js exists with fetch wrapper; but FraudDashboard.jsx never calls any service
C.2	JWT token handling	PARTIALLY WORKING	Token read from localStorage; but set once in constructor — stale if set after init

#	Test	Status	Evidence
C.3	401 error handling	PARTIALLY WORKING	Redirects to /login but also throws — callers may catch the throw
C.4	Loading states	NOT IMPLEMENTED	Frontend has no loading/error UI
C.5	Empty states	NOT IMPLEMENTED	No empty state handling in dashboard
#	Test	Status	Evidence
C.6	HTTP status codes correct	PASS	201 for create, 404 for not found, 400 for bad request, 422 for validation
C.7	Response structure matches schemas	PASS	Pydantic models define all response schemas
C.8	Required fields exist	PASS	All schema fields are defined
C.9	Null/missing value handling	PARTIALLY WORKING	Many nullable fields; some services don't guard against None (e.g., investigation.py:146)
C.10	Invalid data handling	PASS	Pydantic validation raises 422 on invalid input
#	Test	Status	Evidence
C.11	Database connection	PASS	SQLAlchemy async engine with pool_pre_ping=True
C.12	Table creation	PASS	init_db() creates all tables at startup
C.13	Migrations	PASS	Alembic migration 001_initial.py defines all 7 tables

D. Risk Score & Fraud Logic Testing

#	Test	Status	Evidence
D.1	Risk scores 0–100	PARTIALLY WORKING	ML engine normalizes; csv_import.py truncates via int()
D.2	Low (0–30) classification	PARTIALLY WORKING	Threshold exists; magic number 31 used in 4+ files
D.3	Medium (31–70) classification	PARTIALLY WORKING	Same issue as above
D.4	High (71–100) classification	PARTIALLY WORKING	Same issue as above
D.5	Device sharing detection	PASS	NetworkService groups by device_id
D.6	IP sharing detection	PASS	NetworkService groups by ip_address
D.7	Rapid transactions	NOT EXPLICITLY TESTED	No time-window analysis in risk scoring
D.8	Location anomalies	PARTIALLY WORKING	risk_engine.py checks shipping_distance_km > 1000
D.9	Amount anomalies	PARTIALLY WORKING	risk_engine.py checks amount > 10000
D.10	Business rules consistency	FAIL	ml_integration.py has 5 rules; risk_engine.py has 3 rules — scores differ by code path

E. Customer Investigation Testing

#	Test	Status	Evidence
E.1	Find customer	NOT IMPLEMENTED	No customer search in frontend
E.2	View risk profile	PARTIALLY WORKING	Backend endpoint exists; no UI
E.3	View transaction history	PARTIALLY WORKING	Backend endpoint exists; no investigation UI
E.4	View related transactions	PARTIALLY WORKING	Backend aggregates; uses PostgreSQL-specific func.array_agg()

#	Test	Status	Evidence
E.5	View devices	PARTIALLY WORKING	Backend returns device list; no UI
E.6	View IP addresses	PARTIALLY WORKING	Backend returns IP list; no UI
E.7	View locations	PARTIALLY WORKING	Backend returns location list; no UI
E.8	View risk factors	PARTIALLY WORKING	Risk flags in JSON; no UI
E.9	View AI explanation	PARTIALLY WORKING	AI service generates explanations; no UI
E.10	Consistency across screens	NOT TESTABLE	No investigation screens exist

F. AI Investigation Assistant

#	Test	Status	Evidence
F.1	"Why is this customer suspicious?"	PARTIALLY WORKING	Backend can answer; no frontend interface
F.2	"What unusual activity occurred?"	PARTIALLY WORKING	Same
F.3	"Which transactions connected to device?"	PARTIALLY WORKING	Same
F.4	"Summarize investigation"	PARTIALLY WORKING	Endpoint exists; no error fallback
F.5	No hallucinated info	UNTESTABLE	Requires live Gemini API testing
F.6	Error handling on unavailable info	FAIL	generate_investigation_summary and answer_investigation_question have no error fallback — will crash

G. UI / Usability Testing

#	Issue	Severity	Evidence
G.1	All data is hardcoded mock data — no API integration in dashboard	CRITICAL	FraudDashboard.jsx entire file
G.2	Sidebar search input is non-functional — uncontrolled input, no onChange	HIGH	FraudDashboard.jsx:287
G.3	Navigation items are cosmetic only — clicking "Transactions", "Alerts" etc. shows same view	HIGH	FraudDashboard.jsx:167-170
G.4	KPI grid layout broken — 7 cards in a 4-column grid produces uneven rows	MEDIUM	FraudDashboard.jsx:309
G.5	Notification badge hardcoded to "9"	MEDIUM	FraudDashboard.jsx:298
G.6	Badge value "47" hardcoded in nav	LOW	FraudDashboard.jsx:94
G.7	No loading states anywhere	MEDIUM	Entire frontend
G.8	No error states anywhere	MEDIUM	Entire frontend
G.9	No empty states anywhere	MEDIUM	Entire frontend
G.10	No responsive behavior beyond basic Tailwind	LOW	Tailwind breakpoints used
G.11	Google Fonts loaded via CSS @import in JSX — blocking render	LOW	FraudDashboard.jsx:186

#	Issue	Severity	Evidence
G.12	No React StrictMode, no error boundary, no router	MEDIUM	App.jsx

H. Edge Case Testing

#	Edge Case	Status	Evidence
H.1	Customer with no transactions	PASS	Investigation service handles None row (with bug at investigation.py:146)
H.2	Customer with many transactions	PASS	Pagination exists; limit parameter caps results
H.3	Missing device	PASS	device_id is nullable in Transaction model
H.4	Missing IP	PASS	ip_address is nullable
H.5	Missing location	PASS	location is nullable
H.6	Missing risk score	FAIL	Network graph always reads undefined for risk (BUG-001)
H.7	Transaction with invalid fields	PASS	Pydantic validation rejects invalid data
H.8	Duplicate relationships	PARTIALLY WORKING	Network service doesn't deduplicate edges
H.9	Shared device between customers	PASS	Network service creates separate edges per customer
H.10	Shared IP between customers	PASS	Same
H.11	No search results	PASS	Returns empty array
H.12	No alerts	PASS	Returns empty array
H.13	API unavailable	FAIL	No frontend error handling; crashes or shows blank
H.14	Empty API response	PARTIALLY WORKING	Some fallbacks exist; not consistent
H.15	Large graph (2000 nodes)	PARTIALLY WORKING	Limit exists; N+1 query issue in edge building may timeout

I. Security / Basic QA Checks

#	Check	Status	Evidence
I.1	Authentication required	PASS	require_analyst dependency on all endpoints
I.2	Role-based access	PASS	require_role() with ADMIN/MANAGER/ANALYST
I.3	Password hashing	PASS	bcrypt via passlib
I.4	JWT tokens	PASS	HS256 with expiry
I.5	Default secret key	FAIL	config.py defaults to "your-super-secret-key-change-in-production"
I.6	7-day token expiry	MEDIUM	ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 * 7 — unusually long
I.7	No rate limiting	MEDIUM	No rate limiting on any endpoint
I.8	CORS origins	PASS	Specific origins listed, not wildcard
I.9	Sensitive data in error responses	PASS	Generic "Internal server error" returned
I.10	SQL injection	PASS	SQLAlchemy ORM used throughout
I.11	No file size enforcement	MEDIUM	MAX_UPLOAD_SIZE defined but never checked in import_csv
I.12	No token revocation	MEDIUM	No jti claim, no blacklist mechanism

J. Performance Check

#	Check	Status	Evidence
J.1	Dashboard makes 6 sequential queries	MEDIUM	dashboard_stats.py:22-71 — should use asyncio.gather()
J.2	Network graph N+1 query problem	HIGH	network.py:249-272 — individual DB queries per transaction for edges
J.3	No caching anywhere	MEDIUM	Dashboard polled frequently; no Redis/cache integration despite Redis in config
J.4	ilike with % on both sides	MEDIUM	transactions.py:131 — full-text search without trigram index
J.5	Frontend never hits API	PASS (ironically)	Mock data means no performance issues; but also means no real integration
J.6	echo=settings.DEBUG logs all SQL	LOW	database.py:14 — logs sensitive data in debug mode

K. QA Test Case Report

50 test cases are retained from the original audit.

TEST ID	FEATURE	TEST CASE	EXPECTED	ACTUAL	STATUS	SEVERITY
TC-001	Dashboard KPIs	Load dashboard	Real-time stats from API	Hardcoded mock values	FAIL	CRITICAL
TC-002	Transaction List	View transactions	API-fetched data with pagination	Hardcoded mock data	FAIL	CRITICAL
TC-003	Alert Panel	View alerts	API-fetched alerts	Hardcoded mock alerts	FAIL	CRITICAL
TC-004	Network Graph Risk	Display risk on nodes	Colored risk badge from API	Risk is undefined — no badge	FAIL	CRITICAL
TC-005	Network Graph Filter	Filter by risk level	Nodes filtered by risk	All nodes pass (risk is undefined)	FAIL	CRITICAL
TC-006	Network Graph Data	API → Frontend mapping	risk_level mapped correctly	Frontend reads metadata.risk_score which doesn't exist	FAIL	CRITICAL
TC-007	CSV Import	Import CSV	Transactions created with correct types	Boolean fields converted to int instead of bool	FAIL	HIGH
TC-008	CSV Import	ML scoring during import	Score created with valid numeric data	String values passed to ML engine before type conversion	FAIL	HIGH
TC-009	Sidebar Search	Type in sidebar search	Search executes	Input is uncontrolled — typing does nothing	FAIL	HIGH
TC-010	Navigation	Click nav items	Switch between views	All items show same dashboard view	FAIL	HIGH
TC-011	Risk Rules	Business rules consistency	Same score regardless of code path	ml_integration.py has 5 rules; risk_engine.py has 3	FAIL	HIGH
TC-012	AI Assistant	Investigation summary	Graceful error handling	No error fallback — crashes on API failure	FAIL	HIGH
TC-013	Investigation	Customer profile for no transactions	Graceful handling	AttributeError on row.total_txns when row is None	FAIL	HIGH
TC-014	Transaction Create	Create transaction	Returns 201 with transaction data	Works correctly	PASS	—
TC-015	Transaction List	List with filters	Filtered, paginated results	Works correctly	PASS	—
TC-016	Alert List	List alerts	Paginated, filterable	Works correctly	PASS	—
TC-017	Alert Update	Update alert status	Status updated, resolved_at set	Works correctly	PASS	—
TC-018	Network Graph Nodes	Backend creates entity nodes	Nodes for customers, devices, IPs, txns, locations	Works correctly	PASS	—
TC-019	Network Graph Edges	Backend creates relationships	Edges for uses_device, uses_ip, etc.	Works correctly	PASS	—
TC-020	Auth	Login/register	JWT token issued	Works correctly	PASS	—
TC-021	Dashboard Stats	Backend stats endpoint	6 stats queries return data	Works correctly	PASS	—
TC-022	Risk Score	ML scoring	Score 0–100 with risk level	Works correctly (when model loaded)	PASS	—
TC-023	Transaction Batch	Create batch	Transactions created, duplicates skipped	Works correctly	PASS	—
TC-024	Health Check	/health endpoint	Returns status "healthy"	Works correctly	PASS	—
TC-025	Risk Threshold	0–30 = Low, 31–70 = Med, 71–100 = High	Correct classification	Thresholds exist but duplicated as magic numbers	PARTIAL	MEDIUM
TC-026	Network Graph Search	Search by entity name	Nodes filtered by label	Works on text match	PASS	—
TC-027	Network Graph Zoom	Zoom in/out	Graph scales	ReactFlow default — works	PASS	—
TC-028	Network Graph Pan	Drag to pan	Graph moves	ReactFlow default — works	PASS	—
TC-029	Network Graph Minimap	Show minimap	Overview of graph position	Rendered correctly	PASS	—
TC-030	Dashboard Trends	Transaction trend chart	Real data over time	Hardcoded mock array	FAIL	CRITICAL

TEST ID	FEATURE	TEST CASE	EXPECTED	ACTUAL	STATUS	SEVERITY
TC-031	Dashboard Risk Dist	Risk distribution pie chart	Real data	Hardcoded mock array	FAIL	CRITICAL
TC-032	Suspicious Customers	Top suspicious customers	Real API data	Hardcoded mock data	FAIL	CRITICAL
TC-033	CSV Template Download	Download CSV template	File download triggers	Endpoint is POST, should be GET	FAIL	MEDIUM
TC-034	Error Handling	API error response	User-friendly error message	No error UI in frontend	FAIL	MEDIUM
TC-035	Loading State	Data fetching	Spinner/skeleton shown	No loading UI	FAIL	MEDIUM
TC-036	Empty State	No data	Empty state message	No empty state UI	FAIL	MEDIUM
TC-037	Network Graph Missing Data	Graph with partial data	Graceful rendering	Fallback empty arrays — works	PASS	—
TC-038	Network Graph Large Dataset	2000 nodes	Graph renders	Limit exists; N+1 query may timeout	PARTIAL	HIGH
TC-039	Investigation Detail	Get investigation for customer	Full investigation data	Works but PostgreSQL-specific array_agg	PARTIAL	HIGH
TC-040	Investigation Notes	Add notes to alert	Notes appended with timestamp	Works correctly	PASS	—
TC-041	AI Explanation	Generate explanation for flagged txn	Natural language explanation	Works with fallback on error	PASS	—
TC-042	AI Q&A	Answer investigation question	Relevant answer	No error fallback — crashes	FAIL	HIGH
TC-043	Concurrent Operations	Multiple API calls in hooks	State per operation	Shared loading/error — states overwrite	FAIL	MEDIUM
TC-044	Unmount Cleanup	Component unmounts during fetch	No state update on unmounted	No AbortController — React warnings	FAIL	MEDIUM
TC-045	Token Refresh	Token expires	Refresh mechanism	No token refresh — 7-day expiry only	FAIL	MEDIUM
TC-046	File Upload Size	Upload large file	Rejection at limit	MAX_UPLOAD_SIZE never enforced	FAIL	MEDIUM
TC-047	Risk Assessment Enums	Risk level validation	Enum validation	RiskLevel/Decision are plain str subclasses, not enums	FAIL	MEDIUM
TC-048	Dashboard Alerts Filter	Filter alerts by status	Filtered results	Filters after fetch in Python, not in DB	PARTIAL	LOW
TC-049	Alert Service PATCH	Multiple PATCH calls	No data overwrite	update, updateStatus, addNotes overwrite each other	PARTIAL	LOW
TC-050	datetime.utcnow	All datetime usage	Timezone-aware datetimes	Deprecated datetime.utcnow() used in 3+ files	FAIL	LOW

L. Bug Reports

BUG-001 — Network Graph Risk Data Broken End-to-End [CRITICAL]

Field	Value
BUG ID	BUG-001
TITLE	Network Graph Risk Data Broken End-to-End
SEVERITY	CRITICAL
MODULE	Network Graph (Frontend + Backend)
STEPS TO REPRODUCE	1. Load network graph with API data. 2. Observe nodes have no risk badge/color. 3. Apply risk filter — all nodes pass regardless.
EXPECTED	Nodes display colored risk badges (green/amber/red). Risk filter shows only matching nodes.
ACTUAL	Risk is undefined for all API-sourced nodes. No risk badge shown. Risk filter is non-functional.
EVIDENCE	Backend network.py:135 puts only entity_type in metadata. Frontend FraudNetwork.jsx:25 reads node.metadata?.risk_score which doesn't exist. FraudNode.jsx:22-28 reads data.risk which is undefined. theme.js:27 riskTone(undefined) returns default color.
LIKELY OWNER	Frontend API Integration + Backend & DB

BUG-002 — Dashboard Uses Entirely Hardcoded Mock Data [CRITICAL]

Field	Value
BUG ID	BUG-002
TITLE	Dashboard Uses Entirely Hardcoded Mock Data
SEVERITY	CRITICAL
MODULE	Frontend Dashboard
STEPS TO REPRODUCE	1. Open dashboard. 2. Observe KPIs, charts, alerts, tables. 3. All values are static/fake.
EXPECTED	Dashboard fetches real data from API endpoints.
ACTUAL	FraudDashboard.jsx uses hardcoded arrays (trendsData, riskData, suspiciousCustomers, alerts). No service/hook calls.
EVIDENCE	FraudDashboard.jsx:199-258 defines all data inline. No import of services or hooks.
LIKELY OWNER	Frontend API Integration

BUG-003 — CSV Import Passes String Values to ML Engine [HIGH]

Field	Value
BUG ID	BUG-003
TITLE	CSV Import Passes String Values to ML Engine
SEVERITY	HIGH
MODULE	Backend CSV Import
STEPS TO REPRODUCE	1. Import CSV with transaction data. 2. ML scoring is called with txn_data containing string values like "1200.00". 3. ML engine calls float() on strings — may fail or produce incorrect results.
EXPECTED	Type conversion happens before ML scoring.
ACTUAL	ml_service.score_transaction(txn_data) called at csv_import.py:146 before _convert_fields processes numeric/boolean types.
EVIDENCE	csv_import.py:146 calls scoring; _convert_fields runs at lines 162-198 (after scoring).
LIKELY OWNER	Backend & DB

BUG-004 — CSV Import Boolean Fields Converted to int Instead of bool [HIGH]

Field	Value
BUG ID	BUG-004
TITLE	CSV Import Boolean Fields Converted to int Instead of bool
SEVERITY	HIGH

Field	Value
MODULE	Backend CSV Import
STEPS TO REPRODUCE	1. Import CSV with promo_used=true. 2. _convert_fields converts to int(1) at line 184-185. 3. The elif branch at lines 196-198 that converts to bool is never reached.
EXPECTED	Boolean fields stored as Python bool in the database.
ACTUAL	Fields stored as int. The elif logic is dead code.
EVIDENCE	csv_import.py:184-185 converts first; csv_import.py:196-198 never reached.
LIKELY OWNER	Backend & DB

BUG-005 — Business Rules Duplicated with Different Rule Sets [HIGH]

Field	Value
BUG ID	BUG-005
TITLE	Business Rules Duplicated with Different Rule Sets
SEVERITY	HIGH
MODULE	Backend ML Integration + Risk Engine
STEPS TO REPRODUCE	1. Score a transaction with ML engine available → uses 5 rules in ml_integration.py:103-140. 2. Score without ML model → falls back to 3 rules in risk_engine.py:62-70. 3. Same transaction gets different scores.
EXPECTED	Consistent scoring regardless of ML engine availability.
ACTUAL	Two different rule sets produce different risk scores.
EVIDENCE	ml_integration.py has: high_amount (>10000), new_account (<30), failed_cvv, failed_avs, high_risk_country. risk_engine.py has: high_amount (>10000), new_account (<30), failed_cvv+avs. Missing: high_risk_country.
LIKELY OWNER	ML

BUG-006 — Sidebar Search Input Non-Functional [HIGH]

Field	Value
BUG ID	BUG-006
TITLE	Sidebar Search Input Non-Functional
SEVERITY	HIGH
MODULE	Frontend Dashboard
STEPS TO REPRODUCE	1. Click sidebar search input. 2. Type text. 3. Nothing happens — no search executes, no state updates.
EXPECTED	Search input filters data or navigates to search results.
ACTUAL	Input has no value or onChange handler — it's an uncontrolled input that does nothing.
EVIDENCE	FraudDashboard.jsx:287 — <input> element with no event handlers.
LIKELY OWNER	Frontend UI/UX

BUG-007 — Navigation Items Are Non-Functional [HIGH]

Field	Value
BUG ID	BUG-007
TITLE	Navigation Items Are Non-Functional
SEVERITY	HIGH
MODULE	Frontend Dashboard
STEPS TO REPRODUCE	1. Click "Transactions" in sidebar. 2. Click "Alerts" in sidebar. 3. View doesn't change — same dashboard shown.
EXPECTED	Each nav item shows a different view (transaction list, alert list, etc.).
ACTUAL	active state changes highlight but renders identical content. No routing, no conditional rendering.
EVIDENCE	FraudDashboard.jsx:167-170 sets setActive(item) but JSX always renders FraudDashboard content.

Field	Value
LIKELY OWNER	Frontend UI/UX

BUG-008 — Investigation Service Crashes on Customer with No Transactions [HIGH]

Field	Value
BUG ID	BUG-008
TITLE	Investigation Service Crashes on Customer with No Transactions
SEVERITY	HIGH
MODULE	Backend Investigation Service
STEPS TO REPRODUCE	1. Request investigation for a customer_id with no transactions. 2. _get_customer_profile query returns row=None. 3. Line 146 accesses row.total_txns → AttributeError.
EXPECTED	Returns empty/default profile for customer with no transactions.
ACTUAL	Crashes with AttributeError: 'NoneType' object has no attribute 'total_txns'.
EVIDENCE	investigation.py:136 checks row and row.total_txns but lines 142-153 access row fields without null check.
LIKELY OWNER	Backend & DB

BUG-009 — Network Graph N+1 Query Problem []

Field	Value
BUG ID	BUG-009
TITLE	Network Graph N+1 Query Problem
SEVERENCE	HIGH
MODULE	Backend Network Service
STEPS TO REPRODUCE	1. Request network graph with limit=500. 2. Backend executes 1 query for transactions, then N individual queries per transaction for edge building. 3. Response time degrades linearly.
EXPECTED	Batch queries or joined queries to build edges efficiently.
ACTUAL	network.py:249-272 runs individual SELECT per transaction.
EVIDENCE	network.py:249-272 — loop with individual DB queries.
LIKELY OWNER	Backend & DB

BUG-010 — AI Investigation Summary/Q&A Has No Error Fallback [HIGH]

Field	Value
BUG ID	BUG-010
TITLE	AI Investigation Summary/Q&A Has No Error Fallback
SEVERITY	HIGH
MODULE	Backend AI Integration
STEPS TO REPRODUCE	1. Call investigation summary or Q&A endpoint. 2. Gemini API fails (timeout, rate limit, etc.). 3. Exception raised — no fallback explanation returned.
EXPECTED	Graceful fallback message when AI is unavailable.
ACTUAL	generate_investigation_summary and answer_investigation_question don't call _fallback_explanation on error.
EVIDENCE	ai_integration.py:108-136 — no try/except with fallback.
LIKELY OWNER	AI Prompts & Logic

BUG-011 — Frontend Has No Loading, Error, or Empty States []

Field	Value
BUG ID	BUG-011
TITLE	Frontend Has No Loading, Error, or Empty States

Field	Value
SEVERENCE	MEDIUM
MODULE	Frontend Dashboard
STEPS TO REPRODUCE	1. Disconnect network or point to wrong API. 2. Open dashboard. 3. No error message, no spinner, no empty state — blank or stale mock data.
EXPECTED	Loading spinners during fetch, error messages on failure, empty states when no data.
ACTUAL	No loading/error/empty state handling anywhere in FraudDashboard.jsx.
EVIDENCE	Entire FraudDashboard.jsx — no conditional rendering for loading/error states.
LIKELY OWNER	Frontend UI/UX

BUG-012 — All React Hooks Share Single loading/error State [MEDIUM]

Field	Value
BUG ID	BUG-012
TITLE	All React Hooks Share Single loading/error State
SEVERITY	MEDIUM
MODULE	Frontend Hooks
STEPS TO REPRODUCE	1. Call fetchTransactions and createTransaction simultaneously. 2. First completes, sets loading=false. 3. Second is still in progress — but loading is already false.
EXPECTED	Per-operation loading/error states.
ACTUAL	All operations in a hook share one loading and one error state. Concurrent operations overwrite each other.
EVIDENCE	useTransactions.js, useAlerts.js, useDashboard.js, useNetwork.js, useRisk.js — all have single loading/error state.
LIKELY OWNER	Frontend API Integration

BUG-013 — No AbortController / Unmount Cleanup in Hooks []

Field	Value
BUG ID	BUG-013
TITLE	No AbortController / Unmount Cleanup in Hooks
SEVERITY	MEDIUM
MODULE	Frontend Hooks
STEPS TO REPRODUCE	1. Mount component that triggers API call. 2. Unmount before API responds. 3. React warning: "Can't perform a React state update on an unmounted component."
EXPECTED	AbortController cancels fetch on unmount.
ACTUAL	No cleanup in any hook's useEffect return.
EVIDENCE	All hook files — no AbortController or cleanup function.
LIKELY OWNER	Frontend API Integration

BUG-014 — RiskLevel and Decision Not Proper Enums [MEDIUM]

Field	Value
BUG ID	BUG-014
TITLE	RiskLevel and Decision Not Proper Enums
SEVERITY	MEDIUM
MODULE	Backend Schemas
STEPS TO REPRODUCE	1. Call RiskLevel("HIGH") → TypeError. 2. These are str subclasses with class attributes, not enum.Enum.
EXPECTED	Proper enum validation — RiskLevel("HIGH") returns enum member.
ACTUAL	Plain string subclasses — no iteration, no validation.

Field	Value
EVIDENCE	schemas/risk.py:7-16 — class RiskLevel(str): not class RiskLevel(str, Enum):.
LIKELY OWNER	Backend & DB

BUG-015 — datetime.utcnow() Deprecated in Python 3.12+ [MEDIUM]

Field	Value
BUG ID	BUG-015
TITLE	datetime.utcnow() Deprecated in Python 3.12+
SEVERITY	MEDIUM
MODULE	Backend (Multiple Files)
STEPS TO REPRODUCE	1. Run on Python 3.12+. 2. Deprecation warnings generated. 3. Returns naive (non-timezone-aware) datetimes.
EXPECTED	datetime.now(timezone.utc) used throughout.
ACTUAL	datetime.utcnow() used in security.py, investigation.py, alerts.py, dashboard_stats.py, csv_import.py.
EVIDENCE	Multiple files — search for utcnow().
LIKELY OWNER	Backend & DB

BUG-016 — No File Size Enforcement on CSV Upload [MEDIUM]

Field	Value
BUG ID	BUG-016
TITLE	No File Size Enforcement on CSV Upload
SEVERITY	MEDIUM
MODULE	Backend Transactions Router
STEPS TO REPRODUCE	1. Upload a 100MB CSV file. 2. file.read() loads entire file into memory. 3. MAX_UPLOAD_SIZE from config is never checked.
EXPECTED	Reject files exceeding MAX_UPLOAD_SIZE (10MB).
ACTUAL	No size check in import_csv endpoint.
EVIDENCE	transactions.py:160-173 — content = await file.read() with no size check.
LIKELY OWNER	Backend & DB

BUG-017 — Dashboard Makes 6 Sequential Database Queries [MEDIUM]

Field	Value
BUG ID	BUG-017
TITLE	Dashboard Makes 6 Sequential Database Queries
SEVERITY	MEDIUM
MODULE	Backend Dashboard Stats
STEPS TO REPRODUCE	1. Call GET /dashboard/stats. 2. Backend runs 6 sequential queries. 3. Response time = sum of all query times.
EXPECTED	Parallel queries via asyncio.gather() or single combined query.
ACTUAL	dashboard_stats.py:22-71 — sequential await calls.
EVIDENCE	dashboard_stats.py lines 22-71 — 6 separate await calls.
LIKELY OWNER	Backend & DB

BUG-018 — CSV Template Download Endpoint is POST Instead of GET [MEDIUM]

Field	Value
BUG ID	BUG-018
TITLE	CSV Template Download Endpoint is POST Instead of GET

Field	Value
SEVERITY	MEDIUM
MODULE	Backend Transactions Router
STEPS TO REPRODUCE	1. Call POST /import/csv/template. 2. Should be GET — it's a read-only download.
EXPECTED	GET /import/csv/template — semantically correct for download.
ACTUAL	POST /import/csv/template — wrong HTTP method.
EVIDENCE	transactions.py:176 — @router.post("/import/csv/template").
LIKELY OWNER	Backend & DB

BUG-019 — KPI Grid Layout Broken for 7 Cards [LOW]

Field	Value
BUG ID	BUG-019
TITLE	KPI Grid Layout Broken for 7 Cards
SEVERITY	LOW
MODULE	Frontend Dashboard
STEPS TO REPRODUCE	1. View dashboard at xl breakpoint. 2. 7 cards in 4-column grid → 4 on first row, 3 on second row.
EXPECTED	Even layout — either 7-column grid or group into logical sections.
ACTUAL	grid-cols-1 sm:grid-cols-2 xl:grid-cols-4 with 7 children.
EVIDENCE	FraudDashboard.jsx:309.
LIKELY OWNER	Frontend UI/UX

BUG-020 — Notification Badge and Nav Badge Hardcoded [LOW]

Field	Value
BUG ID	BUG-020
TITLE	Notification Badge and Nav Badge Hardcoded
SEVERITY	LOW
MODULE	Frontend Dashboard
STEPS TO REPRODUCE	1. View sidebar. 2. Notification icon shows "9". 3. Badge shows "47". 4. These are static strings.
EXPECTED	Dynamic values from API.
ACTUAL	Hardcoded in JSX.
EVIDENCE	FraudDashboard.jsx:94,298.
LIKELY OWNER	Frontend UI/UX

BUG-021 — get_db Defined in Two Places (DRY Violation) [LOW]

Field	Value
BUG ID	BUG-021
TITLE	get_db Defined in Two Places (DRY Violation)
SEVERITY	LOW
MODULE	Backend Core
STEPS TO REPRODUCE	1. Check database.py:26-31 and deps.py:16-21. 2. Identical implementation in both files.
EXPECTED	Single definition imported where needed.
ACTUAL	Duplicate — maintenance risk if one changes.
EVIDENCE	database.py and deps.py.
LIKELY OWNER	Backend & DB

BUG-022 — Default Secret Key is Insecure Hardcoded String [MEDIUM]

Field	Value
BUG ID	BUG-022
TITLE	Default Secret Key is Insecure Hardcoded String
SEVERITY	MEDIUM
MODULE	Backend Config
STEPS TO REPRODUCE	1. Run without .env file. 2. App starts with SECRET_KEY = "your-super-secret-key-change-in-production".
EXPECTED	App fails to start or uses a generated secure key.
ACTUAL	Insecure default key used silently.
EVIDENCE	config.py:18.
LIKELY OWNER	Backend & DB

BUG-023 — risk_score and customer_behavior_score Always Null [LOW]

Field	Value
BUG ID	BUG-023
TITLE	risk_score and customer_behavior_score Always Null
SEVERITY	LOW
MODULE	Backend Schemas + ML Integration
STEPS TO REPRODUCE	1. Get risk assessment response. 2. customer_behavior_score is always None. 3. RiskAssessment model has the field but nothing sets it.
EXPECTED	Score is computed or field is removed.
ACTUAL	Field exists but is never populated.
EVIDENCE	schemas/risk.py:49-51 — customer_behavior_score: Optional[float] = None. ml_integration.py never sets it.
LIKELY OWNER	ML

M. Final QA Summary

Metric	Count
A. Total Test Cases	50
B. Passed	18
C. Failed	25
D. Partially Passed	7
E. Not Implemented	5 (covered within failed/partial)

Severity	Count	Bug IDs
F. CRITICAL	2	BUG-001, BUG-002
G. HIGH	8	BUG-003, BUG-004, BUG-005, BUG-006, BUG-007, BUG-008, BUG-009, BUG-010
H. MEDIUM	9	BUG-011, BUG-012, BUG-013, BUG-014, BUG-015, BUG-016, BUG-017, BUG-018, BUG-022
LOW	4	BUG-019, BUG-020, BUG-021, BUG-023

Network Graph Status

Area	Status
Backend graph building	PASS — Nodes and edges created correctly
Backend API endpoints	PASS — GET/POST /graph, /clusters functional
Frontend graph rendering	PASS — ReactFlow renders, zoom/pan/minimap work
Frontend search	PASS — Text-based filtering works
Frontend node selection	PASS — Details panel opens
Risk data mapping	CRITICAL FAIL — risk_level not mapped to numeric risk_score; frontend reads nonexistent field
Risk filter	CRITICAL FAIL — Non-functional due to undefined risk values
Risk display	CRITICAL FAIL — No risk badges/colors shown
useMemo dependency	BUG — Missing nodes in dependency array

Overall Project Status

Backend: Solid architecture with functional API endpoints for major features, with several identified quality/performance issues. Frontend: dashboard remains largely mock-data driven and lacks several integration and state-handling features. ML: functional when the model is loaded, with rule consistency and CSV-type issues identified. AI: core Gemini functionality exists, with missing fallbacks in investigation endpoints.

Release Readiness

NOT READY

Critical blockers and recommended priorities are reproduced below from the source audit.

Priority	Team	Issues
1 (immediate)	Frontend API Integration	BUG-002 (mock data), BUG-006 (search), BUG-007 (nav), BUG-011 (loading/error states), BUG-012 (hook state), BUG-013 (unmount cleanup)
2 (immediate)	Network Graph (Frontend + Backend)	BUG-001 (risk data mapping) — needs backend to include risk_score in node metadata, frontend to read it
3 (high)	Backend & DB	BUG-003/004 (CSV import types), BUG-008 (null row crash), BUG-009 (N+1 queries), BUG-016 (file size), BUG-018 (POST→GET), BUG-022 (secret key)
4 (high)	ML	BUG-005 (divergent rules), BUG-023 (null scores)
5 (high)	AI Prompts & Logic	BUG-010 (missing error fallback)
6 (medium)	Frontend UI/UX	BUG-019 (KPI layout), BUG-020 (hardcoded badges)
7 (medium)	Backend & DB	BUG-014 (enums), BUG-015 (utcnow), BUG-017 (sequential queries)